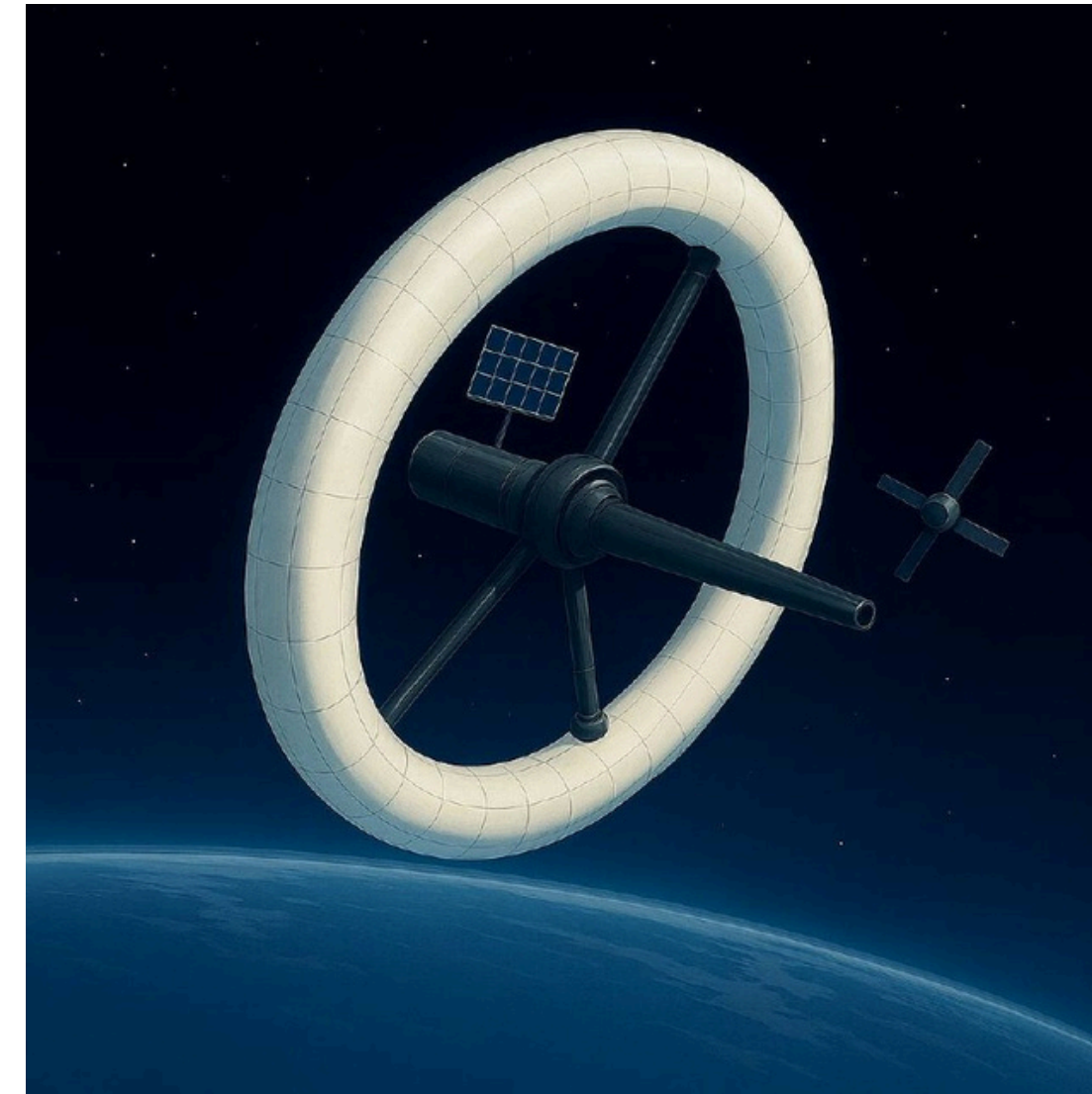




Technical Report

From theory to engineering realism

4.1 Technical Study

1. System Overview

Gravity X is an intelligent embedded system that manages artificial gravity generation and operational control inside the rotating sector of a space station. It integrates AI, sensors, and mechanical control into one cohesive system designed for stability, safety, and energy efficiency.

Main goals:

- Monitor rotational motion and control angular speed.
- Detect obstacles or deviations during internal movement.
- Identify fire or emergencies and trigger visual and voice alerts.
- Operate a smart transition system between fixed and rotating sections.
- Manage internal logistics using centrifugal motion and energy recovery during return trips.

2. Main Components

Hardware

- Main Controller (Arduino / VEX / ESP): Reads sensor data and controls actuators.
- Environmental Sensors: Heat, vibration, flame, and distance (Ultrasonic / IR).
- Rotation Modules: Motors and RPM sensors to track chamber rotation.
- Wireless Communication (Wi-Fi / Bluetooth): Connects the system to the user interface.
- Alert Units (Buzzer + LED): Provide instant visual and audible responses.
- Smart Camera: Analyzes visuals to detect obstacles or humans.
- Data Storage Unit: Logs measurements and system events.

Software

Programmed through multiple environments depending on the controller used:

- PictoBlox / Arduino IDE: Core sensor control and task automation.
- Machine Learning Extension: For hazard classification and state recognition.
- Text-to-Speech & Speech Recognition: For automatic vocal alerts and commands.
- ChatGPT API Integration: Enables AI-based assistance and decision support.
- Data Logging Module: Records telemetry (RPM, distance, temperature, deviations).

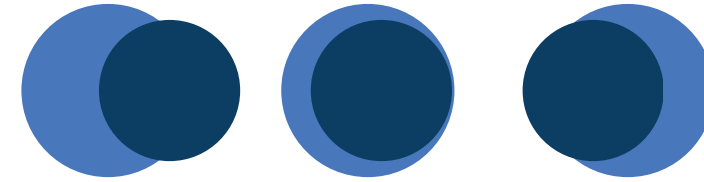
3. Intelligent System Functions (AI & Control)

1. Hazard Detection: Classifies hazards (obstacle, deviation, fire) and severity.
2. Collision Avoidance: Analyzes trajectory and performs auto-rerouting if needed.
3. Fire Detection & Evacuation: Uses camera and heat sensors to trigger CO₂ and audio evacuation.
4. AI Assistant: ChatGPT-based smart assistant for voice commands and system interpretation.
5. Gravity Control: Adjusts rotational speed to maintain artificial gravity balance.
6. Telemetry Monitoring: Displays live values (RPM_A, RPM_B, $\Delta\theta$, Distance, Slot).

4. Reliability & Safety

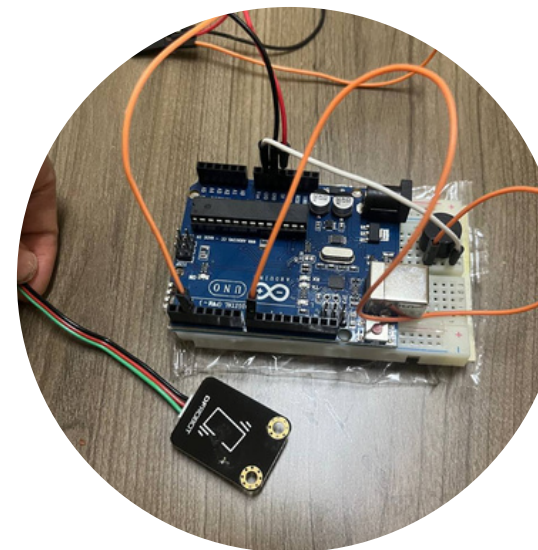
- Equipped with Emergency Stop (E-Stop) for immediate system shutdown.
- CO₂ Discharge used in fire events.
- Implements Fail-Safe algorithms for sensor or communication failure.
- Multi-level alerts (Low / Moderate / High) for hazard prioritization.

4.2 Technical Study Prototype Build



1- Hardware Used

- 2 × VEX EXP robots (merged to form a single structural base).
- 2 DC motors, mounted vertically, to simulate centrifugal force.
- Ultrasonic distance sensor for collision avoidance.
- 2 Arduino boards:
 - o Arduino A: Controls motors, servo, and ultrasonic sensor.
 - o Arduino B: Reads accelerometer/IMU (e.g., MPU-6050) and monitors tilt/acceleration.
- 3D printed capsule for internal chamber simulation.
- Servo motor to open/close the chamber door.
- Motor driver (L298N / TB6612) to handle motor power.
- Breadboard + wiring harness for easy connections.



2-Mechanical Build

- Motors mounted on top of each other to create rotational motion.
- 3D printed capsule attached to simulate an internal living space.
- Ultrasonic sensor mounted on the front for obstacle detection.
- Servo installed inside the capsule as a small access door.



3-Control Logic

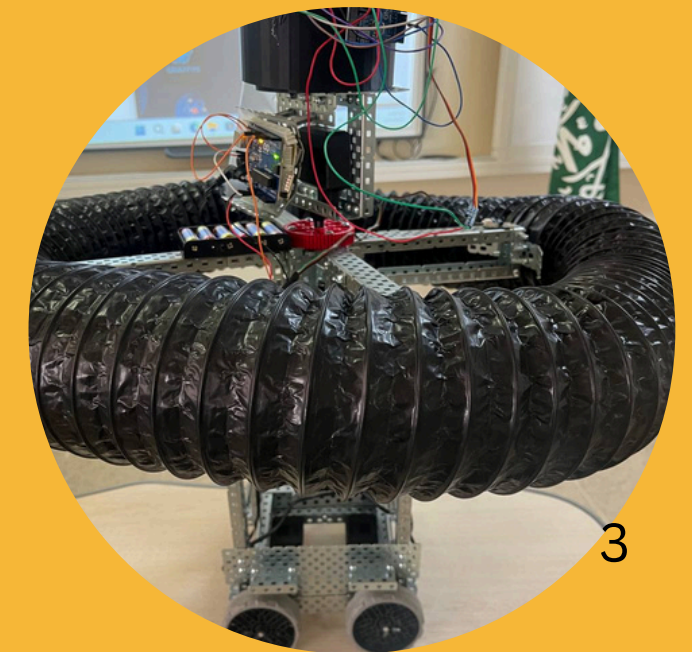
Collision Avoidance: Ultrasonic sensor detects distance → stops motors if object < 25 cm.

Centrifugal Simulation: Motors gradually increase RPM to simulate artificial gravity.

Tilt Monitoring: IMU checks stability; if tilt/acceleration exceeds thresholds → stop command.

Door Control: Servo opens/closes chamber on Arduino command.

AI (PictoBlox): Camera + ML model classifies “safe” vs “dangerous” → sends STOP signal if hazardous object detected.



4.4 Technical Study

The Codes

Programming a VEX Robot to Avoid Collisions

```
from vex import *
import time
brain = Brain()

# ===== Motors (4 wheels) =====
left_front = Motor(Ports.PORT1, GearSetting.RATIO_18_1, False)
left_back = Motor(Ports.PORT2, GearSetting.RATIO_18_1, False)
right_front = Motor(Ports.PORT5, GearSetting.RATIO_18_1, True) # usually right side is
reversed
right_back = Motor(Ports.PORT6, GearSetting.RATIO_18_1, True)
# Fifth motor (always running)
top_motor = Motor(Ports.PORT9, GearSetting.RATIO_18_1, False)

# ===== Sensors (two Distance sensors) =====
front_distance = Distance(Ports.PORT3)
rear_distance = Distance(Ports.PORT7)

# ===== Settings =====
FRONT_NEAR_MM = 220 # if front < 220mm → reverse
FRONT_CLEAR_MM = 280 # only move forward if front > 280mm
REAR_NEAR_MM = 200 # if rear < 200mm → forward
REAR_CLEAR_MM = 260 # only reverse if rear > 260mm
DRIVE_SPEED_PCT = 45
CHECK_INTERVAL_SEC = 0.02
FORWARD_BURST_SEC = 0.50 # forward burst duration (sec)
REVERSE_BURST_SEC = 0.60 # reverse burst duration (sec)

# ===== Helper functions =====
def drive_forward(pct):
    left_front.spin(FORWARD, pct, PERCENT)
    left_back.spin(FORWARD, pct, PERCENT)
    right_front.spin(FORWARD, pct, PERCENT)
    right_back.spin(FORWARD, pct, PERCENT)

def drive_reverse(pct):
    left_front.spin(REVERSE, pct, PERCENT)
    left_back.spin(REVERSE, pct, PERCENT)
    right_front.spin(REVERSE, pct, PERCENT)
    right_back.spin(REVERSE, pct, PERCENT)

def stop_drive():
    left_front.stop()
    left_back.stop()
    right_front.stop()
    right_back.stop()
```

Arduino Code: Triggering an Alarm on Rotation Path Deviation

This example uses a Digital Vibration Sensor V2 with a buzzer.

It triggers a short beep whenever vibration is detected (representing deviation in rotation).

```
// Pin configuration
const uint8_t VIB_PIN = 2; // Sensor signal pin connected to D2
const uint8_t BUZZER = 8; // Buzzer connected to D8

// Beep settings
const unsigned long BEEP_MS = 120; // Beep duration in milliseconds

// Flag for interrupt
volatile bool vibTriggered = false;

// Interrupt Service Routine (ISR) for vibration detection
void vibISR() {
    vibTriggered = true; // Trigger alarm on any vibration pulse
}

void setup() {
    pinMode(VIB_PIN, INPUT_PULLUP); // Most V2 sensors are active LOW
    pinMode(BUZZER, OUTPUT);
    noTone(BUZZER);

    // Use FALLING for active LOW sensors. If no response, try RISING.
    attachInterrupt(digitalPinToInterrupt(VIB_PIN), vibISR, FALLING);
    Serial.begin(9600);

    Serial.println("System ready: Vibration monitoring active...");
}

void loop() {
    if (vibTriggered) {
        vibTriggered = false; // Reset the trigger
        tone(BUZZER, 1500, BEEP_MS); // Short beep
        Serial.println("Vibration detected -> Path deviation alarm!");
    }
}
```

Kinetic-to-Electric Energy

Piezoelectric Sensors

Overview

A piezoelectric sensor (Piezo Disc) converts mechanical energy—such as vibration, pressure, or bending—into electrical energy. This occurs because the piezoelectric material generates a voltage when subjected to mechanical stress.

In the prototype, piezo sensors were attached to the airflow duct to capture vibration produced during system operation. This demonstrates the concept of energy harvesting, similar to Gravity X's approach of converting rotational kinetic energy into electrical power through inertial motion.

Why It Matters

1. Demonstrates Gravity X's Energy-Harvesting Principle

The piezo element provides a small-scale physical model of how Gravity X uses motion and inertia to regenerate power.

2. Vibration Monitoring & Structural Stability

The piezo functions as:

- A vibration detector
- An early-warning indicator for mechanical imbalance
- A real-time sensor for system stability

This mirrors Structural Health Monitoring systems used in rotating space habitats.

3. Adds Scientific Depth to the Prototype

The inclusion of piezoelectric sensors transforms the prototype from a purely mechanical model into a smart, measurable, physics-based system.

```
// Piezoelectric Sensor Reading
const int piezoPin = A0;
int sensorValue = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  sensorValue = analogRead(piezoPin);

  // Print vibration level
  Serial.print("Piezo Output: ");
  Serial.println(sensorValue);

  // Optional: Trigger LED or buzzer on high vibration
  if (sensorValue > 200) {
    Serial.println("High Vibration Detected");
  }

  delay(50);
}
```



4.3 Technical Study

User Interface & Data Flow

1. User Interface (UX/UI)

The Gravity X control system features four main interface panels – Mission Control, Tracking, Hazards, and Transition Chamber – designed for clarity, functionality, and real-time response.

Each interface allows users to monitor mission parameters, control rotational systems, and respond instantly to any detected hazards or deviations.

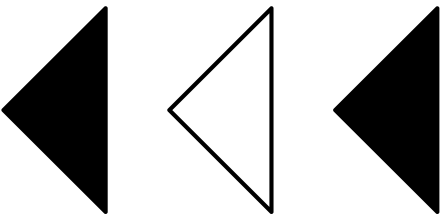
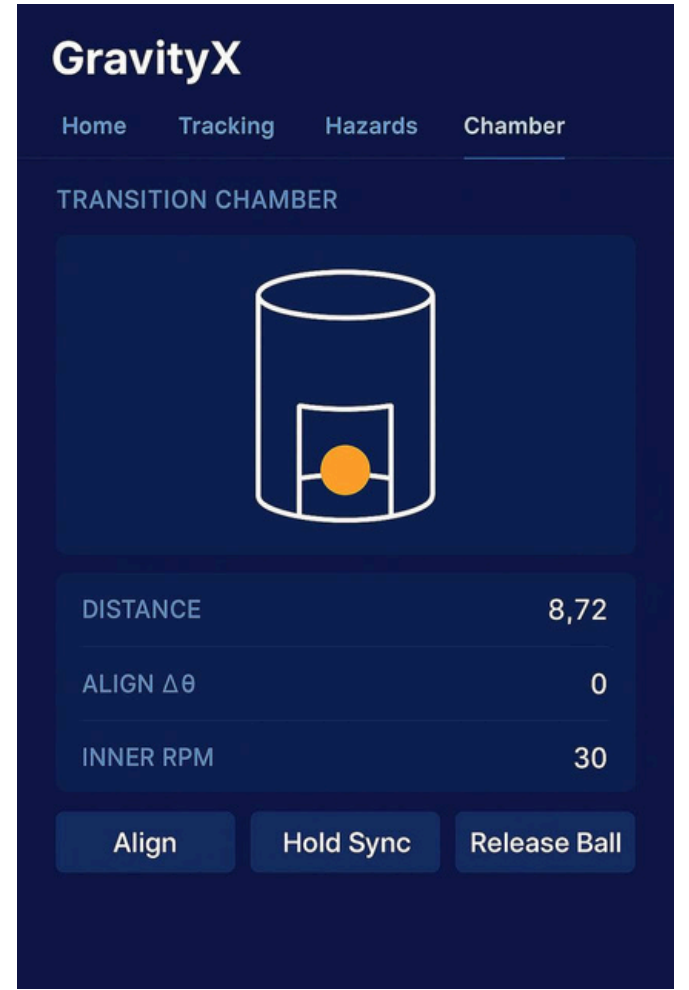
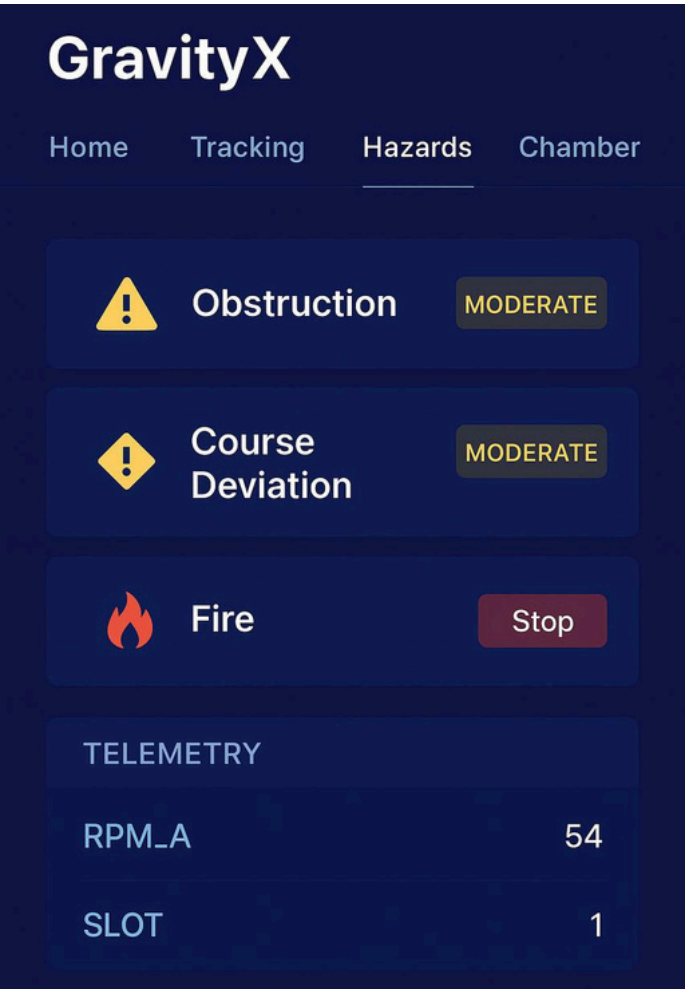
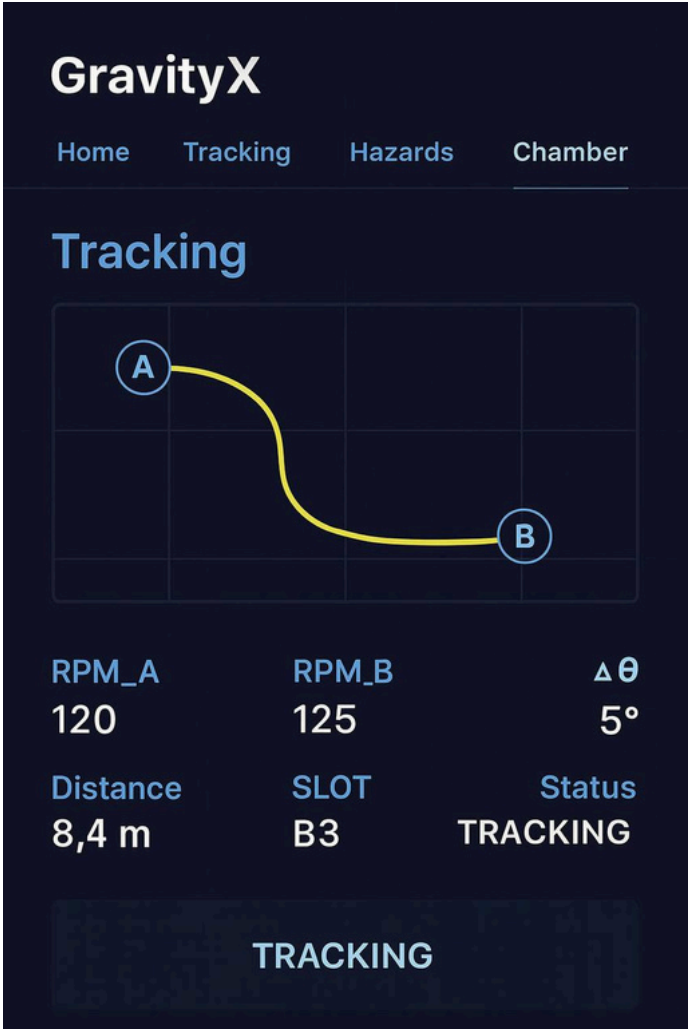
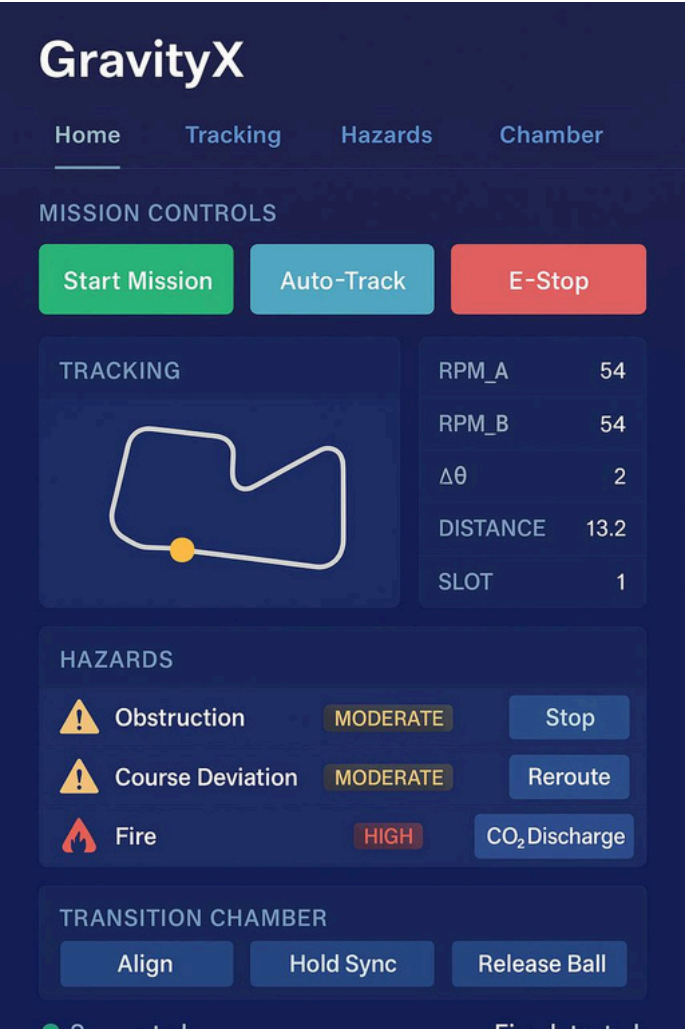
The panels are arranged in a clean grid layout to ensure intuitive navigation and rapid situational awareness.

2. Data Flow Diagram

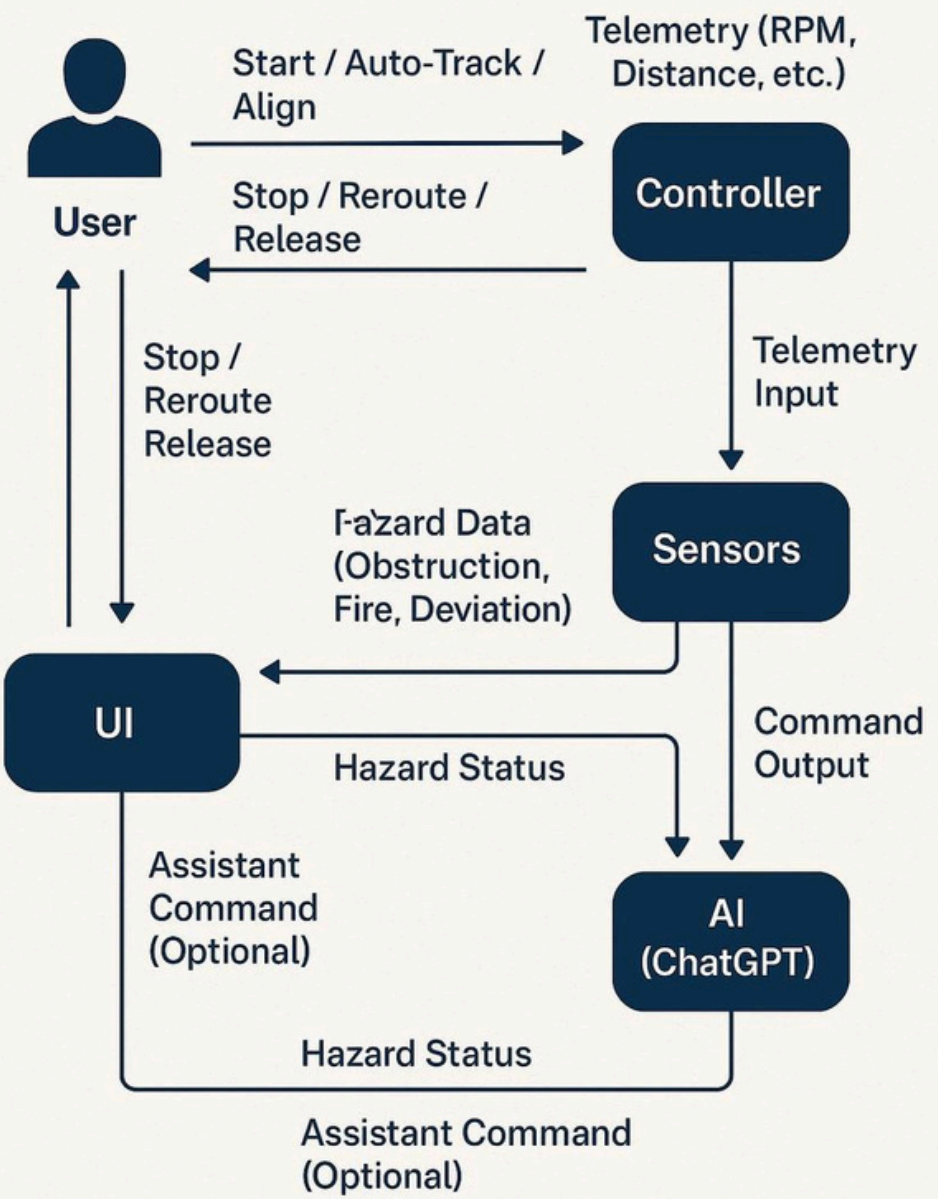
Figure: Data Flow Diagram for the Gravity X Control Interface

This diagram illustrates how user commands and sensor data move through the system – from the interface to the embedded controller, AI assistant, and back to the user via real-time feedback.

It represents the closed communication loop that enables Gravity X to operate as an intelligent embedded system, ensuring continuous monitoring, adaptive control, and instant user feedback.



UI Layer Data Flow Diagram

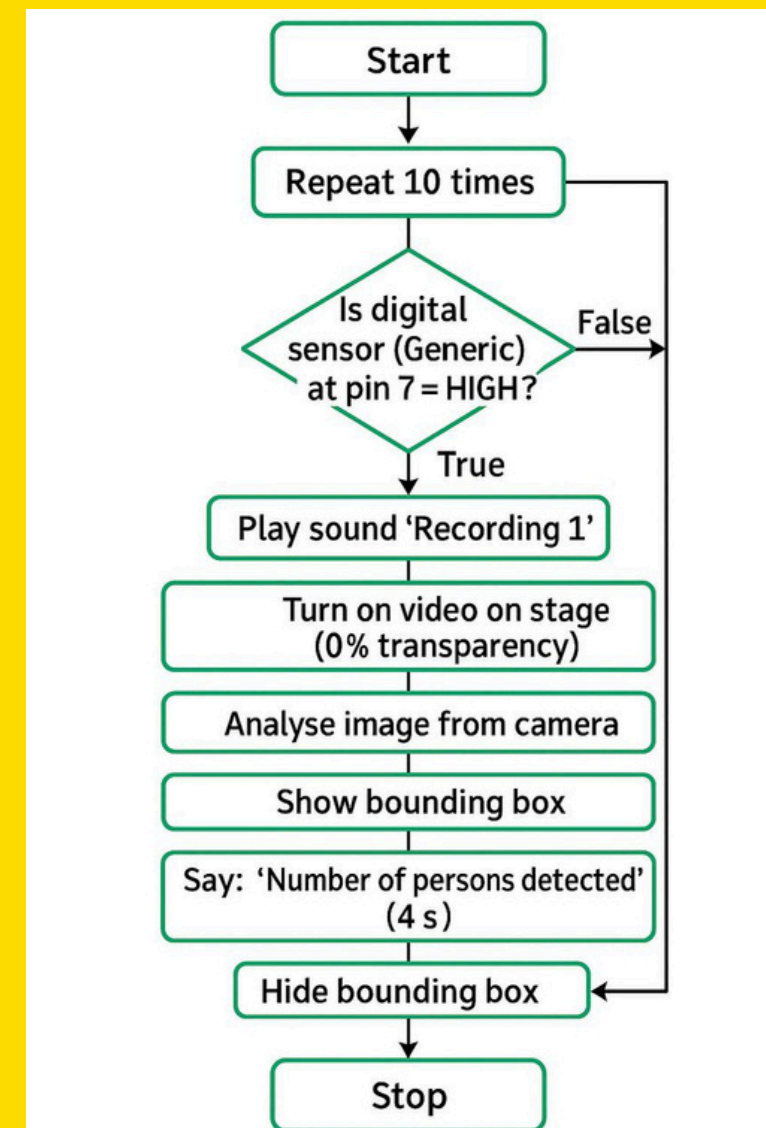
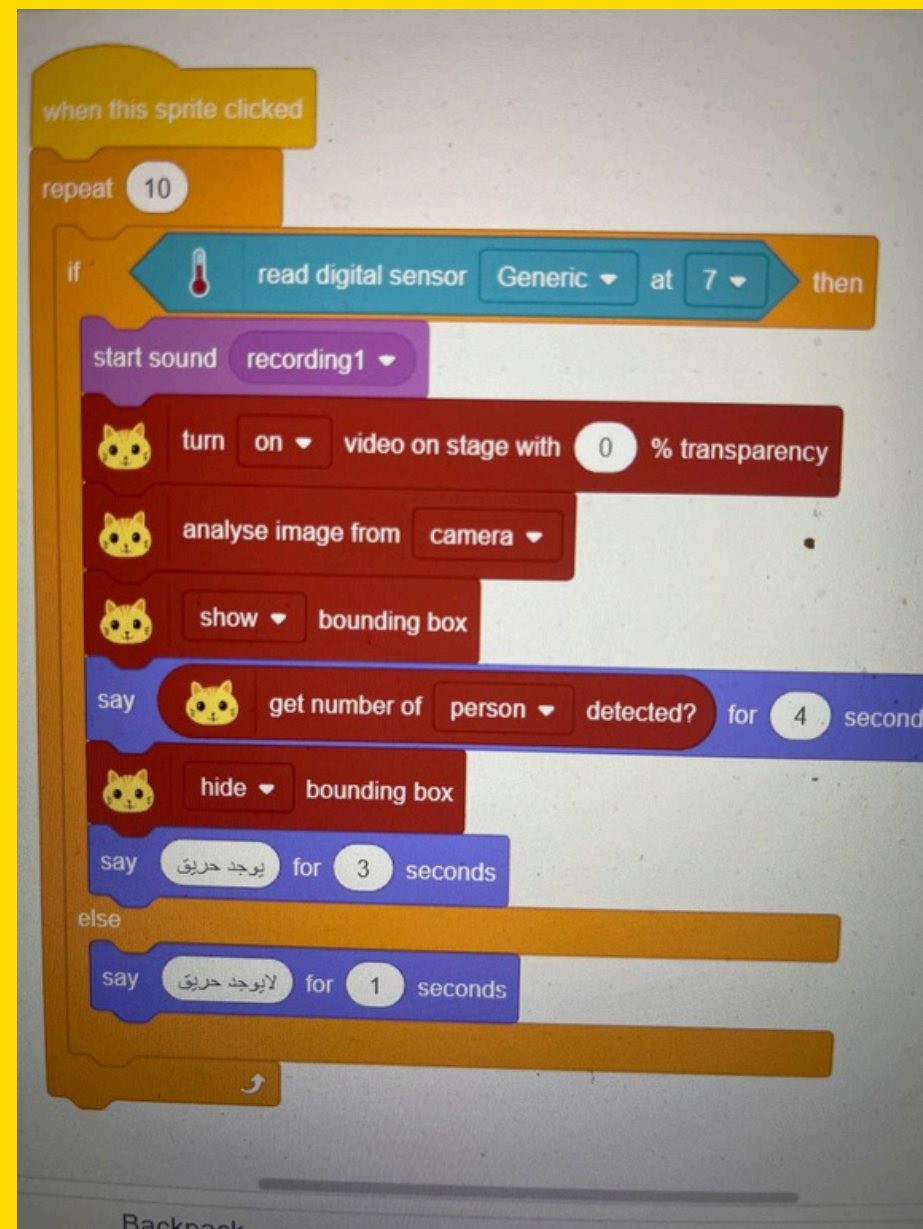


4.4 Technical Study The Codes

“Detection Systems – PictoBlox Logic Flow”



Fire detection system and counting of people in the room with audio alarm



A machine learning-based hazard classification system identifies asteroids, satellites, and space debris.

